

Lista de exercícios

1 Fundamentos

Exercício 1.1

(Solução 1.1)

Assista ao vídeo “*Anything you can do I can do better*” (Marco Lübbecke).

⇒ <https://youtu.be/Dc38La-Xvog>

Exercício 1.2

(Solução 1.2)

Assista ao vídeo “*What’s your problem?*” (Túlio Toffolo).

⇒ <https://youtu.be/ZBA0xKUBvGO>

2 Programação Linear

Exercício 2.1

(Solução 2.1)

Construa, para o modelo Reddy Mikks (estudado em sala de aula), cada uma das seguintes restrições e as expresse com o lado esquerdo linear e com o lado direito constante:

- a) A demanda diária de tinta para interiores ultrapassa a de tinta para exteriores por no mínimo 1.
- b) A utilização diária da matéria-prima $M2$ em toneladas é de no máximo 6 e de no mínimo 3.
- c) A demanda de tinta para interiores não pode ser menor do que a demanda de tinta para exteriores.
- d) A quantidade mínima a ser produzida de ambas as tintas, para interiores e para exteriores, é 3 t.
- e) A proporção de tinta para interiores em relação à produção total de ambas as tintas, para interiores e exteriores, não deve ultrapassar 0,5.

Exercício 2.2

(Solução 2.2)

Determine a melhor solução viável entre as seguintes soluções (viáveis e inviáveis) do modelo Reddy Mikks:

- a) $x_1 = 1, x_2 = 4$
- b) $x_1 = 2, x_2 = 2$
- c) $x_1 = 3, x_2 = 1,5$
- d) $x_1 = 2, x_2 = 1$
- e) $x_1 = 2, x_2 = -1$

Exercício 2.3

(Solução 2.3)

Para a solução viável $x_1 = 2, x_2 = 2$ do modelo Reddy Mikks, determine as quantidades não utilizadas das matérias-primas $M1$ e $M2$.

Exercício 2.4

(Solução 2.4)

Suponha que a Reddy Mikks venda sua tinta para exteriores a um único varejista com um desconto por quantidade. O lucro por tonelada é \$ 5000 se o contratante comprar não mais do que 2 t diárias e, caso contrário, é \$ 4500. Expresse a função objetivo matematicamente. A função resultante é linear?

Exercício 2.5 (Produção)

(Solução 2.5)

Uma empresa que funciona dez horas por dia fabrica dois produtos, os quais passam por três processos de produção. Cada processo é executado em um dia. Logo, cada lote da produção fica pronto em três dias. A tabela abaixo resume os dados do problema.

Produto	Minutos por unidade			Lucro por unidade (\$)
	Processo 1	Processo 2	Processo 3	
1	10	6	8	2
2	5	20	10	3

Modele esse cenário como um problema de otimização linear, com o objetivo de determinar o mix ótimo de produtos para cada três dias de produção.

Exercício 2.6 (FacFactory)

(Solução 2.6)

A FacFactory fabrica dois produtos, A e B. O volume de vendas de A é de no mínimo 80% do total de vendas de ambos (A e B). Contudo, a empresa não pode vender mais do que 100 unidades de A por dia. Ambos os produtos usam uma matéria-prima cuja disponibilidade máxima diária é 240 kg. As taxas de utilização da matéria-prima são 2 kg por unidade de A e 4 kg por unidade de B. Os lucros unitários para A e B são \$ 20 e \$ 50, respectivamente. Construa um programa linear para modelar esse problema e determinar o mix de produto ótimo para a empresa.

Exercício 2.7 (Investidor)

(Solução 2.7)

Um indivíduo quer investir \$ 5.000 no próximo ano em dois tipos de investimento: o investimento A rende 5% e o investimento B rende 8%. Pesquisas de mercado recomendam uma alocação de no mínimo 25% em A e no máximo 50% em B. Além do mais, o investimento em A deve ser no mínimo a metade do investimento em B. Construa o programa linear para calcular o melhor plano de investimento.

Exercício 2.8 (OCC)

(Solução 2.8)

A Divisão de Educação Continuada da Ozark Community College (OCC) oferece um total de 30 cursos a cada semestre. Os cursos oferecidos são, geralmente, de dois tipos: práticos, como de marcenaria, edição de textos e manutenção de carros; e na área de Humanas, como história, música, belas-artes. Para satisfazer as demandas da comunidade, devem ser oferecidos no mínimo dez cursos de cada tipo a cada semestre. A DEC estima que as receitas geradas pelos cursos práticos e da área de Humanas sejam de aproximadamente \$ 1.500 e \$ 1.000 por curso, respectivamente. Elabore um programa linear para o cenário apresentado, com o objetivo de maximizar o lucro da instituição.

Exercício 2.9 (Ulern)

(Solução 2.9)

Jack pretende entrar na Ulern University e já percebeu que “só trabalho e nenhuma diversão faz do Jack um bobalhão”. O resultado é que ele quer partilhar seu tempo disponível de aproximadamente dez horas por dia entre estudo e diversão. Ele estima que se divertir é duas vezes mais interessante do que estudar e, além disso, ele quer estudar pelo menos o mesmo tempo que dedica à diversão. Contudo, Jack percebeu que, se quiser realizar todas as suas tarefas escolares, não pode se divertir mais do que 4 horas por dia. Como ele deve alocar seu tempo para maximizar seu prazer em termos de estudar e se divertir?

Exercício 2.10 (Show & Sell)

(Solução 2.10)

A Show & Sell pode anunciar seus produtos na rádio local e na televisão. A verba de propaganda é limitada a \$ 10.000 por mês. Cada minuto de propaganda pelo rádio custa \$ 15 e cada minuto de comerciais na TV custa \$ 300. A Show & Sell gosta de anunciar pelo rádio no mínimo duas vezes mais do que na TV. Ao mesmo tempo, não é prático usar mais do que 400 minutos por mês de propaganda pelo rádio. Por experiência anterior, a empresa estima que anunciar na TV é 25 vezes mais eficiente

do que anunciar no rádio. Elabore um programa linear para determinar a alocação ótima da verba de propaganda entre rádio e TV.

Exercício 2.11 (Empregos)

(Solução 2.11)

John deve trabalhar no mínimo 20 horas por semana para reforçar sua renda enquanto ainda está na escola. Ele tem a oportunidade de trabalhar em duas lojas de varejo. Na loja 1, pode trabalhar entre 5 e 12 horas por semana, e na loja 2, entre 6 e 10 horas. Ambas pagam o mesmo salário por hora. Para decidir quanto trabalhará em cada loja, John quer basear sua decisão no estresse causado pelo trabalho. Com base em conversas com os atuais empregados, John estima que, em uma escala ascendente de 1 a 10, os fatores de estresse são 8 e 6 nas lojas 1 e 2, respectivamente. Como o estresse aumenta com o tempo, ele considera que o estresse total para cada loja no final da semana seja proporcional ao número de horas que ele trabalhar na loja. Construa um programa linear para decidir quantas horas John deve trabalhar em cada loja.

Exercício 2.12 (OilCo)

(Solução 2.12)

A OilCo está construindo uma refinaria para fabricar quatro produtos: óleo diesel, gasolina, lubrificantes e combustível para jatos. A demanda mínima (em barris/dia) para cada um desses produtos é 14.000, 30.000, 10.000 e 8.000, respectivamente. A OilCo tem contratos de fornecimento de óleo cru pelo Irã e Dubai. Por causa das cotas de produção especificadas pela Organização dos Países Exportadores de Petróleo (Opep), a nova refinaria pode receber no mínimo 40% de seu óleo cru do Irã e a quantidade restante de Dubai. A OilCo prevê que a demanda e as cotas de óleo cru permanecerão estáveis nos próximos dez anos.

As especificações dos dois óleos crus resultam em duas misturas de produtos diferentes: um barril de óleo cru do Irã rende 0,2 barril de diesel, 0,25 barril de gasolina, 0,1 barril de lubrificante e 0,15 barril de combustíveis para jatos; os rendimentos correspondentes do óleo cru de Dubai são 0,1, 0,6, 0,15 e 0,1, respectivamente. A OilCo precisa determinar a capacidade mínima da refinaria (em barris/dia). Elabore o programa linear correspondente.

Exercício 2.13 (Day Trader)

(Solução 2.13)

A Day Trader quer investir uma quantia de dinheiro para gerar um rendimento anual de no mínimo \$ 10.000. Há dois grupos de ações disponíveis: as de primeira linha e as de alta tecnologia, cujos rendimentos médios anuais são 10% e 25%, respectivamente. Embora as ações de empresas de alta tecnologia apresentem um rendimento médio mais alto, são mais arriscadas, e a Trader quer limitar a quantia investida nessas ações a não mais do que 60% do investimento total. Construa um programa linear para determinar a quantidade mínima que a Trader deve investir em cada grupo de ações para cumprir a meta de rendimento do investimento.

Exercício 2.14 (Sucatas)

(Solução 2.14)

Uma central industrial de reciclagem usa dois tipos de sucata de alumínio, A e B, para produzir uma liga especial. A sucata A contém 6% de alumínio, 3% de silício e 4% de carbono. A sucata B tem 3% de alumínio, 6% de silício e 3% de carbono. Os custos por tonelada das sucatas A e B são \$ 100 e \$ 80, respectivamente. As especificações da liga especial requerem que 1) o teor de alumínio deva ser no mínimo 3% e no máximo 6%; 2) o teor de silício deva ficar entre 3% e 5%; e 3) o teor de carbono deva ficar entre 3% e 7%. Formule esse cenário como um problema de otimização linear para determinar o mix ótimo (i.e. de menor custo) de sucatas que deve ser usado para produzir 1.000 toneladas da liga.

Exercício 2.15 (Rádios)

(Solução 2.15)

Uma linha de montagem que consiste em três estações consecutivas produz dois modelos de rádio: HiFi-1 e HiFi-2. A tabela abaixo dá os tempos de montagem para as três estações de trabalho.

Estações de trabalho	Minutos por unidade	
	HiFi-1	HiFi-2
1	6	4
2	5	5
3	4	6

A manutenção diária para as estações 1, 2 e 3 consome 10%, 14% e 12%, respectivamente, de um máximo de 480 minutos disponíveis para cada estação por dia. Elabore um programa linear para determinar o mix ótimo de produtos que minimizará o tempo ocioso (ou não utilizado) nas três estações de trabalho.

Exercício 2.16

(Solução 2.16)

Resolva os modelos construídos nos exercícios anteriores usando o método gráfico. Em particular, encontre a solução ótima e o valor correspondente da função objetivo para os modelos dos seguintes exercícios.

- a) Exercício 2.5 – Produção
- b) Exercício 2.6 – FacFactory
- c) Exercício 2.7 – Investidor
- d) Exercício 2.13 – Day Trader
- e) Exercício 2.14 – Sucatas
- f) Exercício 2.15 – Rádios

Exercício 2.17 (Pyomo)

(Solução 2.17)

Resolva os modelos construídos nos exercícios anteriores usando o *framework* Pyomo e o *solver* GLPK. Em particular, encontre a solução ótima e o valor correspondente da função objetivo para os modelos dos seguintes exercícios.

- a) Exercício 2.8 – OCC
- b) Exercício 2.9 – Ulern
- c) Exercício 2.10 – Show & Sell
- d) Exercício 2.11 – Empregos
- e) Exercício 2.12 – OilCo
- f) Exercício 2.13 – Day Trader

Exercício 2.18

(Solução 2.18)

Construa modelos de programação linear genéricos para os problemas dos exercícios anteriores. Os dados numéricos de cada cenário devem ser representados como parâmetros simbólicos, e não incorporados diretamente nas restrições ou na função objetivo, de modo que o mesmo modelo possa ser reutilizado para resolver diferentes instâncias do problema, bastando alterar os valores desses parâmetros. Para os exercícios 2.5, 2.7 e 2.9, use uma formulação por variáveis. Para os exercícios 2.6, 2.8 e 2.10, use uma formulação por conjuntos.

- a) Exercício 2.5 – Produção
- b) Exercício 2.6 – FacFactory
- c) Exercício 2.7 – Investidor
- d) Exercício 2.8 – OCC
- e) Exercício 2.9 – Ulern
- f) Exercício 2.10 – Show & Sell

Exercício 2.19 (Cervejaria)

(Solução 2.19)

Uma cervejaria precisa planejar a produção de cerveja conforme as encomendas recebidas de clientes. Um pedido é composto pela quantidade de cerveja, em litros, e o percentual de teor alcoólico desejado. A cerveja é produzida misturando diferentes componentes (tipos de cerveja, água, vinho, etc.), cada qual com seu custo por litro e seu teor alcoólico. Dado um pedido, a cervejaria deseja planejar a produção, i.e. quanto de cada componente incluir na mistura, de modo a produzir a cerveja de menor custo e

atendendo os requisitos do cliente. Construa um modelo de programação linear para essa tomada de decisão. Implemente esse modelo usando o *framework* Pyomo e o *solver* GLPK.

Exercício 2.20 (Donovan)

(Solução 2.20)

A Donovan T&T é uma transportadora sediada em Oakwood, nos Estados Unidos. A empresa é especializada no transporte de grãos (milho, soja, feijão, etc.) por caminhões. Recentemente, a diretoria percebeu que sua receita está estagnada, e estuda alternativas para aumentar o faturamento. Uma delas é garantir as melhores escolhas na hora de planejar o transporte a ser realizado por um caminhão. Ajude a empresa nessa tarefa!

O caminhão possui uma capacidade de transporte (em t) e um volume máximo (em m³). Há diferentes tipos de grãos a serem transportados. Cada tipo possui uma densidade (em t/m³), um volume máximo que pode ser transportado (em m³), definido pela legislação do estado, e uma receita esperada (em \$/m³).

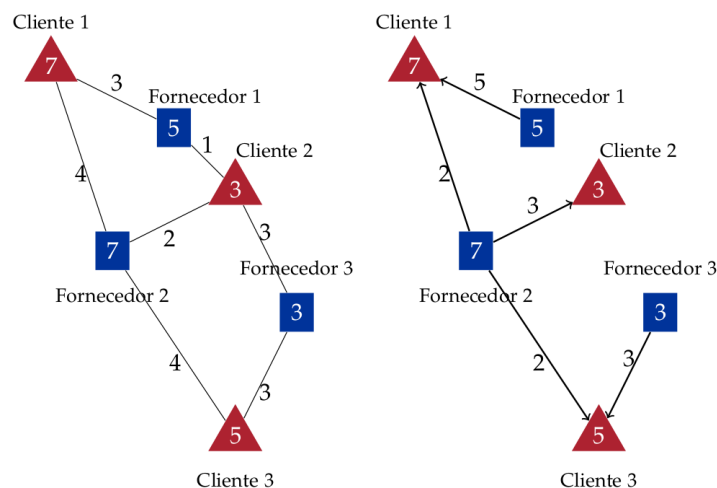
Construa um modelo de programação linear para decidir quanto de cada grão transportar (m³) para um dado caminhão, visando aumentar a receita da empresa. Implemente esse modelo usando o *framework* Pyomo e o *solver* GLPK.

Exercício 2.21 (Transporte)

(Solução 2.21)

Uma empresa tem m depósitos, cada um com estoque a_i , $i = \{1, 2, \dots, m\}$ toneladas de determinado produto. Há pedidos de n clientes de b_j , $j = 1, 2, \dots, n$ toneladas do produto. O transporte de uma tonelada do produto de um depósito i para um cliente j custa c_{ij} . O problema do transporte consiste em determinar o esquema de transporte de menor custo para atender aos pedidos.

Consideremos uma instância concreta do problema, representada pelo grafo abaixo (esquerda). Os valores dos vértices definem o estoque dos depósitos e a demanda dos clientes. Os pesos nas arestas indicam o custo de transporte dos depósitos aos clientes. O lado direito da figura apresenta uma solução para a instância, onde os arcos representam o transporte do produto e seus pesos indicam a quantidade transportada.



Construa um modelo genérico de programação linear para resolver o problema do transporte. Implemente esse modelo usando o *framework* Pyomo e o *solver* GLPK.

Exercício 2.22 (Laboratório transporte)

(Solução 2.22)

Este exercício consiste em um laboratório para o problema do transporte, cujo modelo genérico foi construído no exercício anterior. O arquivo `instance.py`, cujo conteúdo é apresentado abaixo, é um gerador de instâncias para esse problema. Gere 10 instâncias, para valores de semente $\{1, 2, \dots, 10\}$, e as resolva usando sua implementação.

Arquivo `instance.py`:

```
1  import random
2
3  m = None
4  n = None
5  estoque = None
6  demanda = None
7  custos = None
8
9  def gera(semente):
10     global m, n, estoque, demanda, custos
11
12     random.seed(semente)
13     m = random.randint(3, 10)
14     n = random.randint(3, 10)
15
16     estoque = []
17     for i in range(m):
18         estoque.append(random.randint(3, 20))
19
20     demanda = []
21     for j in range(n):
22         demanda.append(random.randint(3, 10))
23
24     custos = []
25     for i in range(m):
26         custos.append([])
27         for j in range(n):
28             custos[i].append(random.randint(1, 5))
29
30
31 def mostra():
32     print()
33     print(f'Fornecedores m = {m}')
34     print(f'Clientes n = {n}')
35     print()
36     print(f'Estoques: {estoque}')
37     print(f'Demandas: {demanda}')
38     print()
39     print(f'Custos:')
40     for l in custos:
41         print(l)
```

Exercício 2.23

(Solução 2.23)

Para praticar e aprofundar, resolva os exercícios de modelagem e solução de problemas de otimização linear dos materiais indicados abaixo.

- **Capítulo 2** de Thie & Keough – *An introduction to linear programming and game theory*, 3 ed., John Wiley & Sons, 2008.
- **Capítulo 2** de Lachtermacher – *Pesquisa operacional na tomada de decisões*, 5 ed., LTC, 2016.
- **Capítulos 2 e 3** de Colin – *Pesquisa operacional: 170 aplicações em estratégia, finanças, logística, produção, marketing e vendas*, LTC, 2007.

3 Programação Inteira

Exercício 3.1 (Localização de instalações capacitado)

(Solução 3.1)

Considere o problema da localização de instalações estudado em sala de aula. Na sua versão capacitada, o problema assume que as instalações possuem uma capacidade de atendimento (constante para qualquer instalação) e as cidades possuem uma demanda. A alocação de instalações a cidades deve respeitar a

capacidade de atendimento. Modifique o modelo matemático e sua implementação para acomodar essas novas características.

Exercício 3.2 (Múltiplas mochilas)

([Solução 3.2](#))

O *problema das múltiplas mochilas* é uma generalização do problema da mochila clássico. Nessa variante, há m mochilas (em vez de somente uma), com capacidades c_j para $j = \{1, 2, \dots, m\}$. A decisão não consiste somente em selecionar ou não cada item, mas também em qual mochila ele será alocado. Formule um modelo linear para resolver esse problema.

Exercício 3.3 (Aplicações)

([Solução 3.3](#))

Uma empresa deve decidir em quais aplicações financeiras deve investir. Há sete opções, cada uma com o lucro estimado e o valor a ser investido conforme tabela abaixo. Cada aplicação pode ser feita somente uma vez.

	Aplicação						
	1	2	3	4	5	6	7
Lucro (\$ $\times$ 1000)	17	10	15	19	7	13	9
Valor (\$ $\times$ 1000)	43	28	34	48	17	32	23

A empresa dispõe de \$ 100 mil de capital disponível para investimento. Formule um modelo de otimização para decidir em quais aplicações investir, maximizando o lucro estimado e respeitando as seguintes restrições: as aplicações $\{1, 2\}$ e $\{3, 4\}$ são mutuamente exclusivas; nem a aplicação 3 e nem a aplicação 4 podem ser feitas, sem ao menos investimento em 1 ou 2; as demais aplicações não têm restrições.

Exercício 3.4 (Coloração de vértices)

([Solução 3.4](#))

Seja $G = (V, E)$ um grafo não-dirigido, uma *coloração* é uma atribuição de cores aos vértices $v \in V$, onde as duas extremidades de cada aresta $e \in E$ têm cores diferentes. Em outras palavras, não deve existir nenhum par de vértices vizinhos com a mesma cor. Queremos colorir o grafo com o menor número de cores possível. Formule e implemente um modelo linear para resolver o problema da coloração de vértices. Use as instâncias disponíveis [nesta página](#) para avaliar o desempenho da implementação em função do tamanho da instância.

Exercício 3.5 (Sudoku)

([Solução 3.5](#))

O *puzzle* japonês [Sudoku](#) pode ser resolvido com programação inteira via coloração de grafos. As cores representam os números de 1 a 9. Cada célula do tabuleiro é representada por um grafo. Vértices que correspondem a células que não podem receber o mesmo número são vizinhos no grafo. A representação abaixo ilustra a construção do grafo.

Use as instâncias disponíveis [nesta página](#). Faça a leitura da instância, a construção do grafo e use seu modelo de coloração de vértices para resolvê-la. Qual o custo computacional? Você consegue projetar um algoritmo mais simples para resolver as instâncias do Sudoku?

Exercício 3.6 (Conjunto dominante)

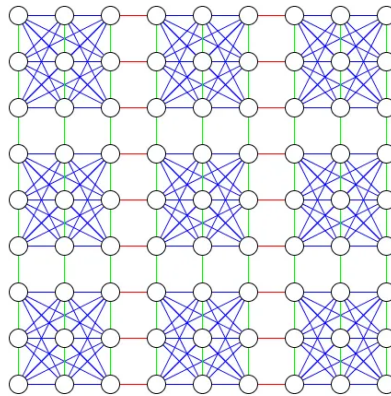
([Solução 3.6](#))

Um *conjunto dominante* de um grafo não-dirigido $G = (V, E)$ é um conjunto $D \subseteq V$, tal que $\forall v \in V : v \in D \vee (\exists u \in D : \{u, v\} \in E)$. Ou seja, cada vértice faz parte do conjunto dominante ou está conectado (i.e. é vizinho) a pelo menos um vértice do conjunto dominante. Estamos interessados em minimizar $|D|$, isto é, em encontrar o menor conjunto dominante do grafo. Formule e implemente um modelo linear para resolver o problema do conjunto dominante. Use as instâncias disponíveis [nesta página](#) para avaliar o desempenho da implementação em função do tamanho da instância.

Exercício 3.7 (Conjunto k-dominante)

([Solução 3.7](#))

Uma generalização do problema do conjunto dominante consiste em encontrar um conjunto tal que cada



vértice que não pertence a ele esteja conectada a pelo menos k vértices do conjunto. Esse problema é conhecido como *conjunto k -dominante*. Modifique o modelo e implementação construídos no exercício anterior para acomodar essa generalização.

Exercício 3.8 (Designação)

(Solução 3.8)

O *problema da designação* (*problema da alocação* ou *assignment problem*) consiste em alocar unidades de produção (e.g. pessoas ou máquinas) a tarefas. Existem n unidades de produção e n tarefas. Uma tarefa i alocada a uma unidade de produção j possui um custo de produção associado c_{ij} . Queremos a alocação de tarefas a unidades de produção, de modo a minimizar o custo de produção total. Uma tarefa deve ser associada a exatamente uma unidade de produção. Uma unidade de produção deve ser associada a exatamente uma tarefa. Formule um modelo linear para resolver o problema da designação.

Exercício 3.9 (Cobertura por arestas)

(Solução 3.9)

Seja $G = (V, E)$ um grafo não-dirigido e ponderado, com pesos $c : E \rightarrow Q$ associados às arestas do grafo, uma *cobertura por arestas* é um subconjunto $E' \subseteq E$ das arestas tal que todo vértice faz parte de pelo menos uma aresta de E' . Queremos minimizar o custo da cobertura, isto é, encontrar a cobertura por arestas cujo somatório de pesos seja minimizado. Formule e implemente um modelo linear para resolver o problema da cobertura por arestas ponderada.

Exercício 3.10 (Clique)

(Solução 3.10)

Seja $G = (V, E)$ um grafo não-dirigido e ponderado, com pesos $c : V \rightarrow Q$ associados aos vértices do grafo, uma *clique* é um subconjunto $V' \subseteq V$ dos vértices tal que existe uma aresta entre todo par de vértices de V' . Queremos minimizar o custo do clique, isto é, encontrar o clique cujo somatório de pesos seja minimizado. Formule e implemente um modelo linear para resolver o problema do clique mínimo ponderado.

Soluções dos exercícios

1 Fundamentos

Solução 1.1

(Exercício 1.1)

Assista ao vídeo sugerido.

Solução 1.2

(Exercício 1.2)

Assista ao vídeo sugerido.

2 Programação Linear

Solução 2.1

(Exercício 2.1)

a) $x_2 - x_1 \geq 1$
 $-x_1 + x_2 \geq 1$

d) $x_1 + x_2 \geq 3$

b) $x_1 + 2x_2 \geq 3$
 $x_1 + 2x_2 \leq 6$

e) $\frac{x_2}{x_1 + x_2} \leq 0,5$

c) $x_2 \geq x_1$
 $-x_1 + x_2 \geq 0$

Solução 2.2

(Exercício 2.2)

Solução (x_1, x_2)	Restrições					Função objetivo ($5x_1 + 4x_2$)
	$6x_1 + 4x_2 \leq 24$	$x_1 + 2x_2 \leq 6$	$-x_1 + x_2 \leq 1$	$x_2 \leq 2$	$x_1, x_2 \geq 0$	
a) (1, 4)	$6 \cdot 1 + 4 \cdot 4 = 22 \leq 24$	$1 + 2 \cdot 4 = 9 > 6$	$-1 + 4 = 3 > 1$	$4 > 2$	$1, 4 \geq 0$	$5 \cdot 1 + 4 \cdot 4 = 21$
b) (2, 2)	$6 \cdot 2 + 4 \cdot 2 = 20 \leq 24$	$2 + 2 \cdot 2 = 6 \leq 6$	$-2 + 2 = 0 \leq 1$	$2 \leq 2$	$2, 2 \geq 0$	$5 \cdot 2 + 4 \cdot 2 = 18$
c) (3, 1,5)	$6 \cdot 3 + 4 \cdot 1,5 = 24 \leq 24$	$3 + 2 \cdot 1,5 = 6 \leq 6$	$-3 + 1,5 = -1,5 \leq 1$	$1,5 \leq 2$	$3, 1,5 \geq 0$	$5 \cdot 3 + 4 \cdot 1,5 = 21$
d) (2, 1)	$6 \cdot 2 + 4 \cdot 1 = 16 \leq 24$	$2 + 2 \cdot 1 = 4 \leq 6$	$-2 + 1 = -1 \leq 1$	$1 \leq 2$	$2, 1 \geq 0$	$5 \cdot 2 + 4 \cdot 1 = 14$
e) (2, -1)	$6 \cdot 2 + 4 \cdot (-1) = 8 \leq 24$	$2 + 2 \cdot (-1) = 0 \leq 6$	$-2 + (-1) = -3 \leq 1$	$1 \leq 2$	$2, -1 \not\geq 0$	$5 \cdot 2 + 4 \cdot (-1) = 6$

Solução 2.3

(Exercício 2.3)

Para $M1$: $6x_1 + 4x_2 = 6 \cdot 2 + 4 \cdot 2 = 20$. Logo, $24 - 20 = 4$ t de sobra.

Para $M2$: $x_1 + 2x_2 = 2 + 2 \cdot 2 = 6$. Logo, $6 - 6 = 0$ t de sobra.

Solução 2.4

(Exercício 2.4)

A função z resultante (abaixo) não é linear.

$$z = \begin{cases} 5x_1 + 4x_2 & , \text{ se } x_1 \leq 2, \\ 4,5x_1 + 4x_2 & , \text{ se } x_1 > 2. \end{cases}$$

Solução 2.5 (Produção)

(Exercício 2.5)

$$\begin{array}{ll}\text{maximiza} & z = 2x_1 + 3x_2 \\ \text{sujeito a} & 10x_1 + 5x_2 \leq 600 \\ & 6x_1 + 20x_2 \leq 600 \\ & 8x_1 + 10x_2 \leq 600 \\ & x_1, x_2 \geq 0\end{array}$$

Solução 2.6 (FacFactory)

(Exercício 2.6)

$$\begin{array}{ll}\text{maximiza} & 20A + 50B \\ \text{sujeito a} & 2A + 4B \leq 240 \\ & A \leq 100 \\ & A \geq 0,8(A + B) \iff 0,2A - 0,8B \geq 0 \\ & A, B \geq 0\end{array}$$

Solução 2.7 (Investidor)

(Exercício 2.7)

$$\begin{array}{ll}\text{maximiza} & z = 0,05A + 0,08B \\ \text{sujeito a} & A + B \leq 5000 \\ & A \geq 0,25(A + B) \iff 0,75A - 0,25B \geq 0 \\ & B \leq 0,5(A + B) \iff -0,5A + 0,5B \leq 0 \\ & A \geq 0,5B \iff A - 0,5B \geq 0 \\ & A, B \geq 0\end{array}$$

Solução 2.8 (OCC)

(Exercício 2.8)

$$\begin{array}{ll}\text{maximiza} & z = 1500x_1 + 1000x_2 \\ \text{sujeito a} & x_1 + x_2 \leq 30 \\ & x_1 \geq 10 \\ & x_2 \geq 10 \\ & x_1, x_2 \geq 0\end{array}$$

Solução 2.9 (Ulern)

(Exercício 2.9)

$$\begin{array}{ll}\text{maximiza} & z = e + 2d \\ \text{sujeito a} & e + d \leq 10 \\ & e \geq d \iff e - d \geq 0 \\ & d \leq 4 \\ & e, d \geq 0\end{array}$$

Solução 2.10 (Show & Sell)

(Exercício 2.10)

$$\begin{array}{ll}\text{maximiza} & z = x_1 + 25x_2 \\ \text{sujeito a} & 15x_1 + 300x_2 \leq 10000 \\ & x_1 \geq 2x_2 \iff x_1 - 2x_2 \geq 0 \\ & x_1 \leq 400 \\ & x_1, x_2 \geq 0\end{array}$$

Solução 2.11 (Empregos)

(Exercício 2.11)

$$\begin{array}{ll}\text{minimiza} & z = 8x_1 + 6x_2 \\ \text{sujeito a} & x_1 \geq 5 \\ & x_1 \leq 12 \\ & x_2 \geq 6 \\ & x_2 \leq 10 \\ & x_1 + x_2 \geq 20 \\ & x_1, x_2 \geq 0\end{array}$$

Solução 2.12 (OilCo)

(Exercício 2.12)

$$\begin{array}{ll}\text{minimiza} & z = x_1 + x_2 \\ \text{sujeito a} & 0,2x_1 + 0,1x_2 \geq 14 \\ & 0,25x_1 + 0,6x_2 \geq 30 \\ & 0,1x_1 + 0,15x_2 \geq 10 \\ & 0,15x_1 + 0,1x_2 \geq 8 \\ & x_1 \geq 0,4(x_1 + x_2) \iff 0,6x_1 - 0,4x_2 \geq 0 \\ & x_1, x_2 \geq 0\end{array}$$

Solução 2.13 (Day Trader)

(Exercício 2.13)

$$\begin{array}{ll}\text{minimiza} & z = x_1 + x_2 \\ \text{sujeito a} & 0,1x_1 + 0,25x_2 \geq 10000 \\ & x_2 \leq 0,6(x_1 + x_2) \iff -0,6x_1 + 0,4x_2 \leq 0 \\ & x_1, x_2 \geq 0\end{array}$$

Solução 2.14 (Sucatas)

(Exercício 2.14)

$$\begin{array}{ll}\text{minimiza} & z = 100x_1 + 80x_2 \\ \text{sujeito a} & 0,06x_1 + 0,03x_2 \geq 0,03 \\ & 0,06x_1 + 0,03x_2 \leq 0,06 \\ & 0,03x_1 + 0,06x_2 \geq 0,03 \\ & 0,03x_1 + 0,06x_2 \leq 0,05 \\ & 0,04x_1 + 0,03x_2 \geq 0,03 \\ & 0,04x_1 + 0,03x_2 \leq 0,07 \\ & x_1 + x_2 = 1 \\ & x_1, x_2 \geq 0\end{array}$$

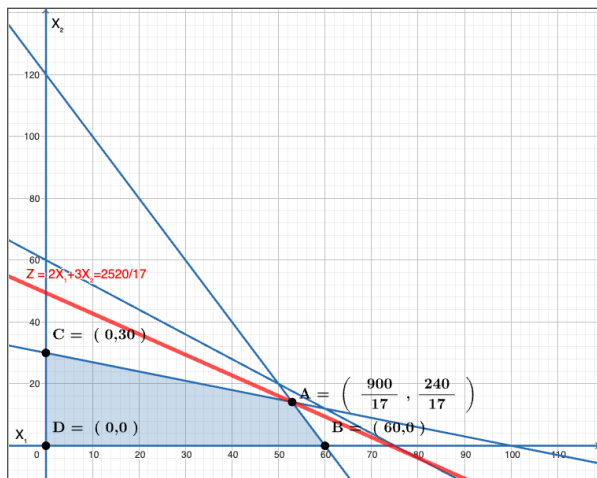
Solução 2.15 (Rádios)

(Exercício 2.15)

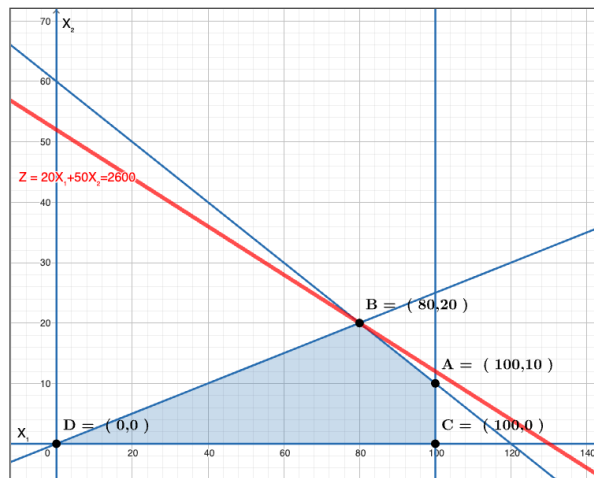
$$\begin{array}{ll}\text{maximiza} & z = 15x_1 + 15x_2 \\ \text{sujeito a} & 6x_1 + 4x_2 \leq 480 \cdot 0,9 \\ & 5x_1 + 5x_2 \leq 480 \cdot 0,86 \\ & 4x_1 + 6x_2 \leq 480 \cdot 0,88 \\ & x_1, x_2 \geq 0\end{array}$$

Solução 2.16

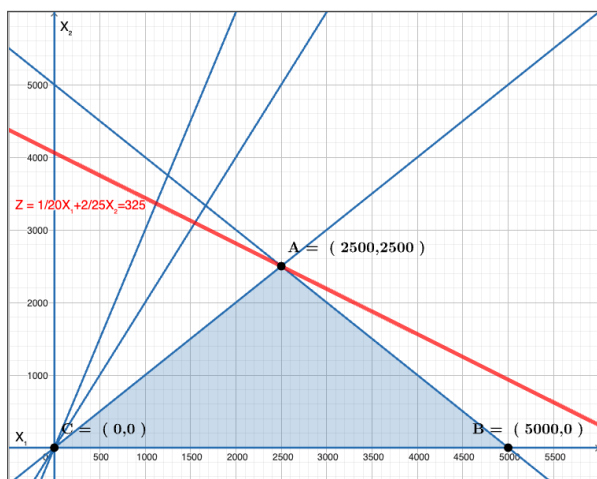
(Exercício 2.16)



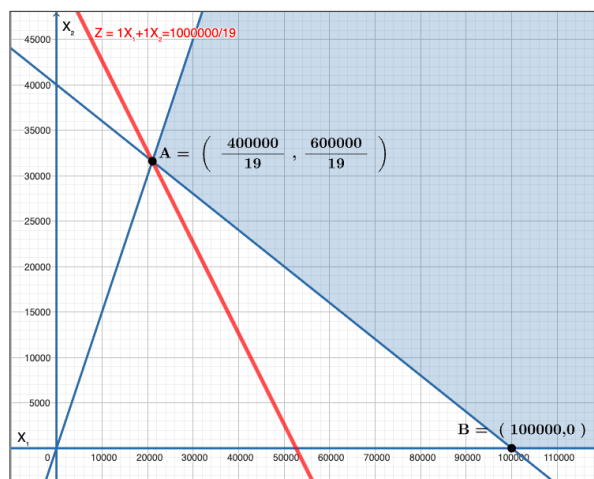
Produção



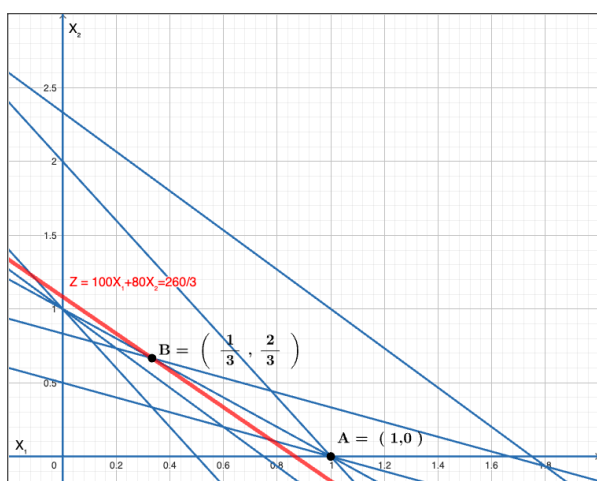
FacFactory



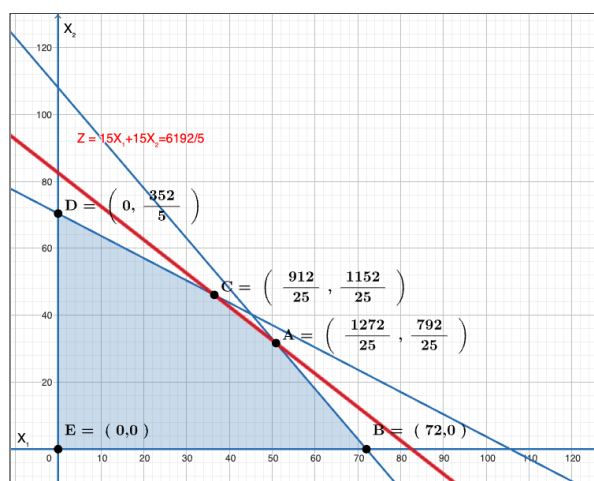
Investidor



Day Trader



Sucatas



Rádio

Solução 2.17 (Pyomo)

(Exercício 2.17)

Veja as implementações em [pyomos.py](#).

OCC:

```
1  from pyomo.environ import *
2
3  model = ConcreteModel()
4
5  model.x = Var([1,2], domain = NonNegativeReals)
6
7  model.obj = Objective(expr = 1500 * model.x[1] + 1000 * model.x[2], sense = maximize)
8
9  model.con1 = Constraint(expr = model.x[1] + model.x[2] <= 30)
10 model.con2 = Constraint(expr = model.x[1] >= 10)
11 model.con3 = Constraint(expr = model.x[2] >= 10)
12
13 opt = SolverFactory('glpk')
14 results = opt.solve(model)
15
16 print('Status:', results.solver.status)
17 print('Termination criterion:', results.solver.termination_condition)
18 if results.solver.termination_condition == 'optimal':
19     print('Optimal solution cost:', model.obj.expr())
20     print('Optimal solution is x1 =', model.x[1].value, 'and x2 =', model.x[2].value)
```

Ulern:

```
1  from pyomo.environ import *
2
3  model = ConcreteModel()
4
5  model.e = Var(domain = NonNegativeReals)
6  model.d = Var(domain = NonNegativeReals)
7
8  def objective_function(model):
9      return model.e + 2 * model.d
10
11 model.obj = Objective(rule = objective_function, sense = maximize)
12
13 def con1(model):
14     return model.e + model.d <= 10
15
16 def con2(model):
17     return model.e - model.d >= 0
18
19 def con3(model):
20     return model.d <= 4
21
22 model.con1 = Constraint(rule = con1)
23 model.con2 = Constraint(rule = con2)
24 model.con3 = Constraint(rule = con3)
25
26 opt = SolverFactory('glpk')
27 opt.solve(model).write()
28 print('\n\nOptimal solution')
29 print('Study:', model.e())
30 print('Fun:', model.d())
31 print('Value:', model.obj())
```

Show & Sell:

```
1  from pyomo.environ import *
2
3  model = ConcreteModel()
4
5  model.x1 = Var(domain = NonNegativeReals)
```

```
6 model.x2 = Var(domain = NonNegativeReals)
7
8 model.obj = Objective(expr = model.x1 + 25 * model.x2, sense = maximize)
9
10 model.con1 = Constraint(expr = 15 * model.x1 + 300 * model.x2 <= 10000)
11 model.con2 = Constraint(expr = model.x1 - 2 * model.x2 >= 0)
12 model.con3 = Constraint(expr = model.x1 <= 400)
13
14 opt = SolverFactory('glpk')
15 opt.solve(model)
16 print('Optimal solution:')
17 print('Radio:', model.x1())
18 print('TV:', model.x2())
19 print('Efficiency:', model.obj())
```

Empregos:

```
1 from pyomo.environ import *
2
3 model = ConcreteModel()
4
5 model.x = Var([1,2], domain = NonNegativeReals)
6
7 model.obj = Objective(expr = 8 * model.x[1] + 6 * model.x[2], sense = minimize)
8
9 model.constraints = ConstraintList()
10 model.constraints.add(expr = model.x[1] >= 5)
11 model.constraints.add(expr = model.x[1] <= 12)
12 model.constraints.add(expr = model.x[2] >= 6)
13 model.constraints.add(expr = model.x[2] <= 10)
14 model.constraints.add(expr = model.x[1] + model.x[2] >= 20)
15
16 opt = SolverFactory('glpk')
17 results = opt.solve(model)
18
19 print('Status:', results.solver.status)
20 print('Termination criterion:', results.solver.termination_condition)
21 if results.solver.termination_condition == 'optimal':
22     print('Total stress:', model.obj.expr())
23     print(f'Opt. solution is {model.x[1].value}h at store 1 and {model.x[2].value}h at store 2!')
```

OilCo:

```
1 from pyomo.environ import *
2
3 model = ConcreteModel()
4
5 model.x = Var([1,2], domain = NonNegativeReals)
6
7 model.obj = Objective(expr = model.x[1] + model.x[2], sense = minimize)
8
9 model.constraints = ConstraintList()
10 model.constraints.add(expr = 0.2 * model.x[1] + 0.1 * model.x[2] >= 14)
11 model.constraints.add(expr = 0.25 * model.x[1] + 0.6 * model.x[2] >= 30)
12 model.constraints.add(expr = 0.1 * model.x[1] + 0.15 * model.x[2] >= 10)
13 model.constraints.add(expr = 0.15 * model.x[1] + 0.1 * model.x[2] >= 8)
14 model.constraints.add(expr = 0.6 * model.x[1] - 0.4 * model.x[2] >= 0)
15
16 opt = SolverFactory('glpk')
17 opt.solve(model).write()
18 print('\n\nOptimal solution')
19 print('x1:', model.x[1]())
20 print('x2:', model.x[2]())
21 print('Value:', model.obj())
```

Day Trader:

```
1 from pyomo.environ import *
2
3 model = ConcreteModel()
4
5 model.x = Var([1,2], domain = NonNegativeReals)
6
7 model.obj = Objective(expr = model.x[1] + model.x[2], sense = minimize)
8
9 model.constraints = ConstraintList()
10 model.constraints.add(expr = 0.1 * model.x[1] + 0.25 * model.x[2] >= 10000)
11 model.constraints.add(expr = -0.6 * model.x[1] + 0.4 * model.x[2] <= 0)
12
13 opt = SolverFactory('glpk')
14 opt.solve(model).write()
15 print('\n\nOptimal solution')
16 print('x1:', model.x[1]())
17 print('x2:', model.x[2]())
18 print('Value:', model.obj())
```

Solução 2.18

(Exercício 2.18)

Abaixo são apresentados os modelos genéricos.

PRODUÇÃO

Dados:

- n é o número de produtos;
- m é o número de processos de produção;
- T é o tempo disponível por dia em cada processo (em minutos);
- t_{ij} é o tempo (em minutos) exigido pelo produto $i = \{1, \dots, n\}$ no processo $j = \{1, \dots, m\}$;
- L_i é o lucro por unidade produzida do produto $i = \{1, \dots, n\}$.

Variáveis de decisão:

- x_i : quantidade de unidades produzidas do produto $i = \{1, \dots, n\}$.

Modelo genérico:

$$\begin{array}{ll} \text{maximiza} & \sum_{i=1}^n L_i x_i \\ \text{sujeito a} & \sum_{i=1}^n t_{ij} x_i \leq T, \quad j = \{1, \dots, m\} \\ & x_i \geq 0, \quad i = \{1, \dots, n\} \end{array}$$

FACFACTORY

Dados:

- P é o conjunto de produtos;
- Q é a disponibilidade máxima diária de matéria-prima (em kg);
- r_p é a taxa de utilização de matéria-prima (em kg) por unidade do produto $p \in P$;

- L_p é o lucro unitário do produto $p \in P$;
- f_p é a fração mínima exigida das vendas do produto $p \in P$ em relação ao total vendido;
- U_p é o limite máximo de vendas diárias do produto $p \in P$.

Variáveis de decisão:

- x_p : quantidade vendida do produto $p \in P$.

Modelo genérico:

$$\begin{aligned}
 &\text{maximiza} && \sum_{p \in P} L_p x_p \\
 &\text{sujeito a} && \sum_{p \in P} r_p x_p \leq Q \\
 &&& x_p \leq U_p, && p \in P \\
 &&& x_p \geq f_p \sum_{q \in P} x_q, && p \in P \\
 &&& x_p \geq 0, && p \in P
 \end{aligned}$$

INVESTIDOR

Dados:

- n é o número de tipos de investimento;
- M é o capital total disponível para investir;
- r_i é a taxa de retorno do investimento $i = \{1, \dots, n\}$;
- f_i é a fração mínima do capital total que deve ser alocada ao investimento $i = \{1, \dots, n\}$;
- g_i é a fração máxima do capital total que pode ser alocada ao investimento $i = \{1, \dots, n\}$;
- a_{ij} é a razão mínima exigida entre o investimento $i = \{1, \dots, n\}$ e o investimento $j = \{1, \dots, n\}$ (o valor investido em i deve ser no mínimo a_{ij} vezes o valor investido em j).

Variáveis de decisão:

- x_i : valor investido no tipo de investimento $i = \{1, \dots, n\}$.

Modelo genérico:

$$\begin{aligned}
 &\text{maximiza} && \sum_{i=1}^n r_i x_i \\
 &\text{sujeito a} && \sum_{i=1}^n x_i \leq M \\
 &&& x_i \geq f_i \sum_{j=1}^n x_j, && i = \{1, \dots, n\} \\
 &&& x_i \leq g_i \sum_{j=1}^n x_j, && i = \{1, \dots, n\} \\
 &&& x_i \geq a_{ij} x_j, && i, j = \{1, \dots, n\} \\
 &&& x_i \geq 0, && i = \{1, \dots, n\}
 \end{aligned}$$

OCC

Dados:

- T é o conjunto de tipos de curso;
- C é o número total de cursos oferecidos por semestre;
- R_t é a receita gerada por um curso do tipo $t \in T$;
- D_t é o número mínimo de cursos do tipo $t \in T$ que devem ser oferecidos por semestre.

Variáveis de decisão:

- x_t : número de cursos do tipo $t \in T$ oferecidos por semestre.

Modelo genérico:

$$\begin{array}{ll}\text{maximiza} & \sum_{t \in T} R_t x_t \\ \text{sujeito a} & \sum_{t \in T} x_t \leq C \\ & x_t \geq D_t, \quad t \in T \\ & x_t \geq 0, \quad t \in T\end{array}$$

ULERN

Dados:

- n é o número de atividades;
- T é o tempo total disponível por dia;
- p_i é o grau de prazer proporcionado por uma hora dedicada à atividade $i = \{1, \dots, n\}$;
- a_{ij} é a razão mínima exigida entre o tempo dedicado à atividade $i = \{1, \dots, n\}$ e o tempo dedicado à atividade $j = \{1, \dots, n\}$ (o tempo em i deve ser no mínimo a_{ij} vezes o tempo em j);
- U_i é o tempo máximo diário que pode ser dedicado à atividade $i = \{1, \dots, n\}$.

Variáveis de decisão:

- x_i : tempo (em horas) dedicado à atividade $i = \{1, \dots, n\}$.

Modelo genérico:

$$\begin{array}{ll}\text{maximiza} & \sum_{i=1}^n p_i x_i \\ \text{sujeito a} & \sum_{i=1}^n x_i \leq T \\ & x_i \geq a_{ij} x_j, \quad i, j = \{1, \dots, n\} \\ & x_i \leq U_i, \quad i = \{1, \dots, n\} \\ & x_i \geq 0, \quad i = \{1, \dots, n\}\end{array}$$

SHOW & SELL

Dados:

- M é o conjunto de mídias de propaganda;
- B é a verba total disponível para propaganda por mês;
- c_m é o custo por minuto de propaganda na mídia $m \in M$;
- E_m é a eficiência de propaganda na mídia $m \in M$;
- a_{mn} é a razão mínima exigida entre o tempo de propaganda na mídia $m \in M$ e na mídia $n \in M$ (o tempo em m deve ser no mínimo a_{mn} vezes o tempo em n);
- U_m é o tempo máximo mensal (em minutos) de propaganda na mídia $m \in M$.

Variáveis de decisão:

- x_m : tempo (em minutos) de propaganda na mídia $m \in M$ por mês.

Modelo genérico:

$$\begin{array}{ll} \text{maximiza} & \sum_{m \in M} E_m x_m \\ \text{sujeito a} & \sum_{m \in M} c_m x_m \leq B \\ & x_m \geq a_{mn} x_n, \quad m, n \in M \\ & x_m \leq U_m, \quad m \in M \\ & x_m \geq 0, \quad m \in M \end{array}$$

Solução 2.19 (Cervejaria)

(Exercício 2.19)

Veja a implementação em [cervejaria.py](#).

Dados:

- V é o volume do pedido (em litros de cerveja);
- $\bar{A}$ é o teor alcoólico desejado;
- C é o conjunto de componentes (tipos de cerveja, água, ...).
- P_c é o preço de cada componente $c \in C$;
- A_c é o teor alcoólico de cada componente $c \in C$.

Variáveis de decisão:

- x_c : quantidade (litros) de cada componente $c \in C$ na mistura.

Modelo genérico:

$$\begin{aligned} &\text{minimiza} && \sum_{c \in C} x_c P_c \\ &\text{sujeito a} && \sum_{c \in C} x_c = V \\ & && \sum_{c \in C} x_c A_c = \bar{A}V \\ & && x_c \geq 0, \quad \forall c \in C \end{aligned}$$

Implementação:

```
1  from pyomo.environ import *
2
3  # Volume do pedido
4  V = 100
5
6  # Teor alcoólico desejado
7  A = 0.06
8
9  components = {
10     'Cerveja A': {'A': 0.058, 'P': 0.28},
11     'Cerveja B': {'A': 0.037, 'P': 0.25},
12     'Água':      {'A': 0.000, 'P': 0.05},
13     'Vinho':     {'A': 0.083, 'P': 0.40}
14 }
15
16 # Lista de componentes
17 C = components.keys()
18
19 # Modelo
20 model = ConcreteModel()
21
22 # Variáveis de decisão: uma para cada componente
23 model.x = Var(C, domain = NonNegativeReals)
24
25 # Função objetivo
26 model.cost = Objective(expr = sum(model.x[c] * components[c]['P'] for c in C))
27
28 # Restrições
29 model.vol = Constraint(expr = sum(model.x[c] for c in C) == V)
30 model.alc = Constraint(expr = sum(model.x[c] * components[c]['A'] for c in C) == A * V)
31
32 # Solução
33 solver = SolverFactory('glpk')
34 solver.solve(model)
35
36 print('Mistura ótima')
37 for c in C:
38     print('>', c, ': ', model.x[c](), 'litros')
39 print()
40 print('Volume =', model.vol(), 'litros')
41 print('Custo = $', model.cost())
```

Solução 2.20 (Donovan)

(Exercício 2.20)

Veja a implementação em [donovan.py](#).

Dados:

- m tipos de grão;
- T : capacidade (massa) do caminhão (t);
- V : capacidade (volume) do caminhão (m^3);
- D_i : densidade do grão $i = [m]$ (t/m^3);
- Q_i : volume máximo do grão $i = [m]$ (m^3);
- R_i : receita esperada do grão $i = [m]$ ($\$/\text{m}^3$).

Variáveis de decisão:

- x_i : quantidade (volume, i.e. m^3) de transporte de cada grão $i = \{1, 2, \dots, m\}$.

Modelo genérico:

$$\begin{aligned}
 &\text{maximiza} && \sum_{i=1}^m x_i R_i \\
 &\text{sujeito a} && \sum_{i=1}^m x_i \leq V \\
 & && \sum_{i=1}^m x_i D_i \leq T \\
 & && x_i \leq Q_i, && i = \{1, 2, \dots, m\} \\
 & && x_i \geq 0, && i = \{1, 2, \dots, m\}
 \end{aligned}$$

Implementação:

```

1  from pyomo.environ import *
2
3  m = 10
4  T = 28
5  V = 83
6  D = [1.3, 0.7, 1.8, 1.1, 1.0, 1.5, 0.9, 0.8, 1.0, 1.2]
7  Q = [10, 12, 40, 21, 5, 5, 8, 17, 6, 9]
8  R = [5, 3, 2.2, 4, 1.8, 4.1, 3.7, 1.9, 6, 2]
9
10 model = ConcreteModel()
11 model.x = Var(range(m), domain = NonNegativeReals)
12 model.obj = Objective(expr = sum(model.x[i]*R[i] for i in range(m)), sense = maximize)
13
14 model.cons = ConstraintList()
15 model.cons.add(expr = sum(model.x[i] for i in range(m)) <= V)
16 model.cons.add(expr = sum(model.x[i]*D[i] for i in range(m)) <= T)
17 for i in range(m):
18     model.cons.add(expr = model.x[i] <= Q[i])
19
20 solver = SolverFactory('glpk')
21 solver.solve(model).write()
22
23 print('Função objetivo:', model.obj())
24 for i in range(m):
25     print(f'Grão {i + 1}: {model.x[i]() } m3.')
```

Solução 2.21 (Transporte)

(Exercício 2.21)

Seja x_{ij} a variável de decisão que determina a quantidade a ser transportada do depósito i ao cliente j . O modelo genérico para o problema do transporte é apresentado abaixo.

$$\begin{aligned} &\text{minimiza} && \sum_{i \in [m], j \in [n]} c_{ij} x_{ij} \\ &\text{sujeito a} && \sum_{j \in [n]} x_{ij} \leq a_i, \quad \text{para todo fornecedor } i \in [m] \\ & && \sum_{i \in [m]} x_{ij} = b_j, \quad \text{para todo cliente } j \in [n] \\ & && x_{ij} \geq 0, \quad \text{para todo fornecedor } i \in [m] \text{ e todo cliente } j \in [n] \end{aligned}$$

A implementação desse modelo é fornecida abaixo (veja também em [transporte.py](#)).

```
1  from pyomo.environ import *
2
3  m = 3                # fornecedores
4  n = 3                # clientes
5  a = [5, 7, 3]        # estoques
6  b = [7, 3, 5]        # demandas
7  c = [[3, 1, 100],    # custos
8        [4, 2, 4],
9        [100, 3, 3]]
10
11 # Criação do modelo
12 model = ConcreteModel()
13
14 # Variáveis de decisão: para cada par depósito x cliente
15 model.x = Var([i for i in range(m)], [j for j in range(n)], domain = NonNegativeReals)
16
17 def objective_function(model):
18     result = 0
19     for i in range(m):
20         for j in range(n):
21             result += model.x[i,j] * c[i][j]
22     return result
23
24 # Função objetivo
25 model.obj = Objective(rule = objective_function)
26
27 # Restrições
28 model.cons = ConstraintList()
29
30 for i in range(m):
31     model.cons.add(expr = sum(model.x[i,j] for j in range(n)) <= a[i])
32
33 for j in range(n):
34     model.cons.add(expr = sum(model.x[i,j] for i in range(m)) == b[j])
35
36 # Solução
37 solver = SolverFactory('glpk')
38 solver.solve(model).write()
39
40 print('Custo total:', model.obj())
41 for i in range(m):
42     for j in range(n):
43         print(i + 1, '-->', j + 1, ': ', model.x[i,j]())
```

Solução 2.22 (Laboratório transporte)

(Exercício 2.22)

A implementação abaixo recebe o valor de semente na sua invocação (e.g. `python3 model.py 1`), usa o gerador fornecido para gerar a instância correspondente e a resolve usando a implementação do modelo genérico. Veja também em [lab-transporte](#).

```
1  from pyomo.environ import *
2  import instance
3  import sys
4
```

```
5  semente = int(sys.argv[1])
6
7  def mostra_solucao(model):
8      print()
9      print(f'Instância (semente): {semente}')
10
11     print(' ', end = '')
12     for j in range(instance.n):
13         print(f' {j+1:5} ', end = '')
14     print()
15     print(' ' + '-' * (instance.n * 7 + 1))
16
17     for i in range(instance.m):
18         print(f'{i+1} | ', end = '')
19         for j in range(instance.n):
20             print(f' {model.x[i,j]():5} ', end = '')
21         print()
22     print()
23     print('Custo total:', model.obj())
24
25
26 def resolve():
27     instance.gera(semente)
28
29     # Criação do modelo
30     model = ConcreteModel()
31
32     # Variáveis de decisão: para cada par depósito x cliente
33     model.x = Var([i for i in range(instance.m)], [j for j in range(instance.n)], domain =
        ↳ NonNegativeReals)
34
35     def objective_function(model):
36         result = 0
37         for i in range(instance.m):
38             for j in range(instance.n):
39                 result += model.x[i,j] * instance.custos[i][j]
40         return result
41
42     # Função objetivo
43     model.obj = Objective(rule = objective_function)
44
45     # Restrições
46     model.cons = ConstraintList()
47
48     for i in range(instance.m):
49         model.cons.add(expr = sum(model.x[i,j] for j in range(instance.n)) <= instance.estoque[i])
50
51     for j in range(instance.n):
52         model.cons.add(expr = sum(model.x[i,j] for i in range(instance.m)) == instance.demanda[j])
53
54     # Solução
55     solver = SolverFactory('glpk')
56     results = solver.solve(model)
57
58     if results.solver.termination_condition == 'optimal':
59         mostra_solucao(model)
60     else:
61         print('Solução ótima não encontrada!')
62
63
64 resolve()
```

Solução 2.23

(Exercício 2.23)

Exercício de aprofundamento.

3 Programação Inteira

Solução 3.1 (Localização de instalações capacitado)

(Exercício 3.1)

Seja $C \in \mathbb{Z}_+$ a capacidade de atendimento das instalações e $d_i \in \mathbb{Z}_+$, a demanda das cidades $i \in [n]$. Precisamos garantir que nenhuma instalação tenha sua capacidade de atendimento ultrapassada. Isso pode ser feito incluindo o conjunto de restrições

$$\sum_{i \in [n]} d_i x_{ij} \leq C, \quad j \in [n].$$

Podemos ainda assumir que o custo de atendimento é proporcional à demanda da cidade atendida. Neste caso, a função objetivo pode ser substituída por:

$$\text{minimiza} \quad \sum_{j \in [n]} f_j y_j + \sum_{i, j \in [n]} d_i c_{ij} x_{ij}.$$

Solução 3.2 (Múltiplas mochilas)

(Exercício 3.2)

Definimos variáveis de decisão $x_{ij} \in \{0,1\}$ que assumem valor 1 se um item i é alocado à mochila j , e valor 0 caso contrário. O modelo matemático é definido como:

$$\begin{aligned} \text{maximiza} \quad & \sum_{j=1}^m \sum_{i=1}^n v_i x_{ij} \\ \text{sujeito a} \quad & \sum_{i=1}^n p_i x_{ij} \leq c_j, & j = \{1, 2, \dots, m\} \\ & \sum_{j=1}^m x_{ij} \leq 1, & i = \{1, 2, \dots, n\} \\ & x_{ij} \in \{0, 1\}, & i = \{1, 2, \dots, n\}, j = \{1, 2, \dots, m\} \end{aligned}$$

Solução 3.3 (Aplicações)

(Exercício 3.3)

Definimos variáveis de decisão $x_i \in \{0,1\}$ que indicam o investimento ou não em cada aplicação $i = \{1, 2, \dots, 7\}$. O modelo matemático é definido como:

$$\begin{aligned} \text{maximiza} \quad & 17x_1 + 10x_2 + 15x_3 + 19x_4 + 7x_5 + 13x_6 + 9x_7 \\ \text{sujeito a} \quad & 43x_1 + 28x_2 + 34x_3 + 48x_4 + 17x_5 + 32x_6 + 23x_7 \leq 100 \\ & x_1 + x_2 \leq 1 \\ & x_3 + x_4 \leq 1 \\ & x_3 \leq x_1 + x_2 \\ & x_4 \leq x_1 + x_2 \\ & x_i \in \{0, 1\}, & i = \{1, 2, \dots, 7\} \end{aligned}$$

Solução 3.4 (Coloração de vértices)

(Exercício 3.4)

Seja o grafo $G = (V, E)$, de vértices V e arestas E , e C o número máximo de cores. Definimos as variáveis de decisão $x_{vi}, y_i \in \{0, 1\}$, para $v \in V$ e $i \in [C]$. A variável x_{vi} assume valor 1 se o vértice v recebe a cor i ; assume valor 0, caso contrário. A variável y_i assume valor 1 se a cor i é usada; assume valor 0, caso contrário. O modelo matemático é definido como:

$$\begin{aligned}
 &\text{minimiza} && \sum_{i=1}^T y_i \\
 &\text{sujeito a} && \sum_{i=1}^T x_{vi} = 1, && \forall v \in V \\
 &&& x_{ui} + x_{vi} \leq y_i, && \forall \{u, v\} \in E, i = \{1, 2, \dots, C\} \\
 &&& x_{vi}, y_i \in \{0, 1\}, && \forall v \in V, i = \{1, 2, \dots, C\}
 \end{aligned}$$

Solução 3.5 (Sudoku)

(Exercício 3.5)

Exercício de implementação e avaliação experimental. Você pode consultar [este material](#).

Solução 3.6 (Conjunto dominante)

(Exercício 3.6)

Seja o grafo $G = (V, E)$, de vértices V e arestas E , $N(v)$ é o conjunto de vizinhos do vértice $v \in V$. Definimos variáveis de decisão $x_v \in \{0, 1\}$ que indicam se um vértice $v \in V$ faz parte do conjunto dominante (1) ou não (0). O modelo matemático é definido como:

$$\begin{aligned}
 &\text{minimiza} && \sum_{v \in V} x_v \\
 &\text{sujeito a} && x_v + \sum_{u \in N(v)} x_u \geq 1, && \forall v \in V \\
 &&& x_v \in \{0, 1\}, && \forall v \in V
 \end{aligned}$$

Solução 3.7 (Conjunto k-dominante)

(Exercício 3.7)

Basta adicionarmos a constante k no conjunto de restrições.

$$\begin{aligned}
 &\text{minimiza} && \sum_{v \in V} x_v \\
 &\text{sujeito a} && kx_v + \sum_{u \in N(v)} x_u \geq k, && \forall v \in V \\
 &&& x_v \in \{0, 1\}, && \forall v \in V
 \end{aligned}$$

Solução 3.8 (Designação)

(Exercício 3.8)

Seja $x_{ij} \in \{0,1\}$ variáveis de decisão que determinam se uma tarefa $i = \{1, 2, \dots, n\}$ é alocada à unidade de produção $j = \{1, 2, \dots, n\}$ ou não. O modelo matemático é definido como:

$$\begin{aligned}
 &\text{minimiza} && \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\
 &\text{sujeito a} && \sum_{j=1}^n x_{ij} = 1, && i = \{1, 2, \dots, n\} \\
 &&& \sum_{i=1}^n x_{ij} = 1, && j = \{1, 2, \dots, n\} \\
 &&& x_{ij} \in \{0, 1\}, && i, j = \{1, 2, \dots, n\}
 \end{aligned}$$

Solução 3.9 (Cobertura por arestas)

(Exercício 3.9)

Seja o grafo $G = (V, E)$, de vértices V e arestas E , c_{uv} é o custo da aresta $\{u, v\} \in E$, e $N(v)$ é o conjunto de vizinhos do vértice $v \in V$. Definimos variáveis de decisão $x_{uv} \in \{0,1\}$ que assume valor 1 se $\{u, v\} \in E'$ ou valor 0, se $\{u, v\} \notin E'$. O modelo matemático é definido como:

$$\begin{aligned}
 &\text{minimiza} && \sum_{\{u,v\} \in E} c_{uv} x_{uv} \\
 &\text{sujeito a} && \sum_{u \in N(v)} x_{uv} \geq 1, && \forall v \in V \\
 &&& x_{uv} \in \{0, 1\}, && \forall \{u, v\} \in E
 \end{aligned}$$

Solução 3.10 (Clique)

(Exercício 3.10)

Seja o grafo $G = (V, E)$, de vértices V e arestas E , c_v é o custo do vértice $v \in V$. Definimos variáveis de decisão $x_v \in \{0,1\}$ que indicam se o vértice $v \in V$ faz parte do clique ou não. O modelo matemático é definido como:

$$\begin{aligned}
 &\text{minimiza} && \sum_{v \in V} c_v x_v \\
 &\text{sujeito a} && x_u + x_v \leq 1, && \forall \{u, v\} \notin E \\
 &&& x_v \in \{0, 1\}, && \forall v \in V
 \end{aligned}$$